Constraint Satisfaction Problems

Artificial Intelligence — Computer Science — Sofia University

Bogomil Stoyanov

© 2026 Bogomil Stoyanov

Outline

1. Looking inside the state
2. Backtracking search
3. Ordering: which variable, which value
4. Constraint propagation
5. Structure
6. Local search: min-conflicts
7. Summary

© 2026 Bogomil Stoyanov

Looking inside the state

© 2026 Bogomil Stoyanov

A state stops being a black box

Constraint satisfaction problem

- variables $X_1, \ldots, X_n$;
- a domain D_i of allowed values for each;
- constraints, each forbidding some combinations of values.

A **solution** is a complete assignment violating no constraint.

© 2026 Bogomil Stoyanov

A state stops being a black box

Constraint satisfaction problem

- variables $X_1, \ldots, X_n$;
- a domain D_i of allowed values for each;
- constraints, each forbidding some combinations of values.

A **solution** is a complete assignment violating no constraint.

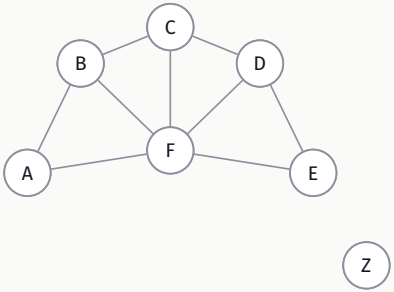
Compare with the last two weeks:

- the goal test is no longer a black box. It is a *conjunction of constraints* we can inspect;
- all solutions sit at depth exactly n;
- the path is irrelevant.

Being able to read the goal test is worth an enormous amount. When a partial assignment already breaks a constraint, we can discard *every* completion of it: billions of states ruled out without generating one of them.

© 2026 Bogomil Stoyanov

Example: graph colouring



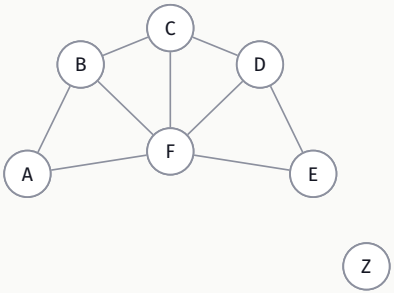
Variables: A, B, C, D, E, F, Z.

Domains: { } for each.

Constraints: adjacent nodes differ, one binary constraint per edge, 9 in all.

© 2026 Bogomil Stoyanov

Example: graph colouring



Variables: A, B, C, D, E, F, Z.

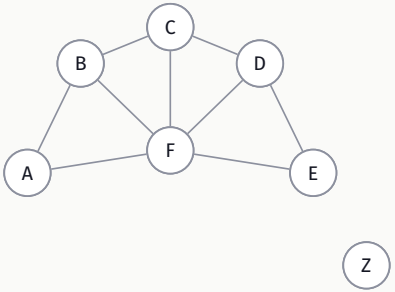
Domains: { } for each.

Constraints: adjacent nodes differ, one binary constraint per edge, 9 in all.

The picture *is* the constraint graph: a node per variable, an edge per binary constraint. Everything we do today is reading structure off this graph.

© 2026 Bogomil Stoyanov

Example: graph colouring



Variables: A, B, C, D, E, F, Z.

Domains: { } for each.

Constraints: adjacent nodes differ, one binary constraint per edge, 9 in all.

The picture *is* the constraint graph: a node per variable, an edge per binary constraint. Everything we do today is reading structure off this graph.

$3^7 = 2187$ assignments. Brute force is fine here, and hopeless one slide from now.

© 2026 Bogomil Stoyanov

Example: Sudoku

5	3			7				
6			1	9	5			
	9	8					6	
8				6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

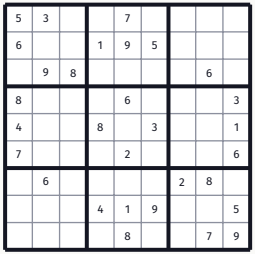
Variables: the 81 cells.

Domains: $\{1..9\}$; a given cell has a domain of size 1.

Constraints: 27 **alldiff** constraints, one per row, per column and per 3×3 block.

© 2026 Bogomil Stoyanov

Example: Sudoku



Variables: the 81 cells.

Domains: $\{1..9\}$; a given cell has a domain of size 1.

Constraints: 27 **alldiff** constraints, one per row, per column and per 3×3 block.

> What a human solver does: never guess-and-check, but "this cell can only be a 4". That is **constraint propagation**, and it is the main subject of this session. An easy Sudoku is one that propagation alone finishes.

© 2026 Bogomil Stoyanov

Not all constraints are binary

Unary: restricts one variable.
$C \neq$ ■ : just delete the value from D_C.

Binary: two variables. An edge in the constraint graph.

Higher-order: three or more. Drawn as a hyperedge, or decomposed into binary constraints with auxiliary variables.

© 2026 Bogomil Stoyanov

Not all constraints are binary

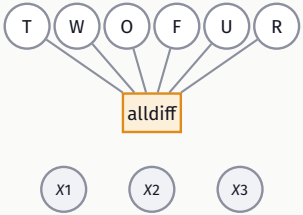
Unary: restricts one variable.
$C \neq$ ■ : just delete the value from D_C.

Binary: two variables. An edge in the constraint graph.

Higher-order: three or more. Drawn as a hyperedge, or decomposed into binary constraints with auxiliary variables.

Constraints may also be **soft**: preferences with a cost rather than a prohibition. That turns the problem into constrained optimisation.

TWO + TWO = FOUR



carry variables turn the column sums into binary constraints

© 2026 Bogomil Stoyanov

Backtracking search

© 2026 Bogomil Stoyanov

Ordinary search is hopeless here

Treat it as last week's search problem: a state is a partial assignment, an action adds one variable.

Then the branching factor at depth ℓ is $(n - \ell)d$, and the tree has

$$n! \cdot d^n \quad \text{leaves.}$$

For Sudoku that is about 10^{197}.

© 2026 Bogomil Stoyanov

Ordinary search is hopeless here

Treat it as last week's search problem: a state is a partial assignment, an action adds one variable.

Then the branching factor at depth ℓ is $(n - \ell)d$, and the tree has

$$n! \cdot d^n \quad \text{leaves.}$$

For Sudoku that is about 10^{197}.

But assignments are **commutative**: A=🟥 then B=🟩 is the same state as B=🟩 then A=🟥.

© 2026 Bogomil Stoyanov

Ordinary search is hopeless here

Treat it as last week's search problem: a state is a partial assignment, an action adds one variable.

Then the branching factor at depth ℓ is $(n - \ell)d$, and the tree has

$$n! \cdot d^n \quad \text{leaves.}$$

For Sudoku that is about 10^{197}.

But assignments are **commutative**: A=🟥 then B=🟩 is the same state as B=🟩 then A=🟥.

So consider only *one* variable per level. The tree collapses from $n!\,d^n$ leaves to d^n, and which variable we pick at each level is free for us to choose. That single observation is **backtracking search**, and it is the base every other technique today builds on.

© 2026 Bogomil Stoyanov

Ordinary search is hopeless here

Treat it as last week's search problem: a state is a partial assignment, an action adds one variable.

Then the branching factor at depth ℓ is $(n - \ell)d$, and the tree has

$$n! \cdot d^n \quad \text{leaves.}$$

For Sudoku that is about 10^{197}.

But assignments are **commutative**: A=■ then B=■ is the same state as B=■ then A=■.

So consider only *one* variable per level. The tree collapses from $n!\,d^n$ leaves to d^n, and which variable we pick at each level is free for us to choose. That single observation is **backtracking search**, and it is the base every other technique today builds on.

Plain backtracking already solves n-queens for $n \approx 25$. We want $n = 10\,000$.

© 2026 Bogomil Stoyanov

Backtracking

BACKTRACK(ASSIGNMENT, CSP)

```
1  if assignment is complete then return assignment
2  X ← SELECT-UNASSIGNED-VARIABLE (csp)
3  for each v in ORDER-DOMAIN-VALUES (X)
4      if v is consistent with assignment
5          add X = v to assignment
6          inference ← PROPAGATE(csp, X, v)
7              r ← BACKTRACK(assignment, csp)
8              if r ≠ failure then return r
9          undo X = v and inference
10 return failure
```

© 2026 Bogomil Stoyanov

Backtracking

BACKTRACK(ASSIGNMENT, CSP)

```
1  if assignment is complete then return assignment
2  X ← SELECT-UNASSIGNED-VARIABLE (csp)
3  for each v in ORDER-DOMAIN-VALUES (X)
4      if v is consistent with assignment
5          add X = v to assignment
6          inference ← PROPAGATE(csp, X, v)
7              r ← BACKTRACK(assignment, csp)
8              if r ≠ failure then return r
9          undo X = v and inference
10 return failure
```

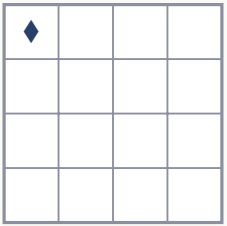
Depth-first search with a consistency check at line 4. That is all.

The three highlighted lines are the only places where anything interesting can happen, and each is the subject of one section below:

- which variable next?
- which value first?
- what can we deduce now?

© 2026 Bogomil Stoyanov

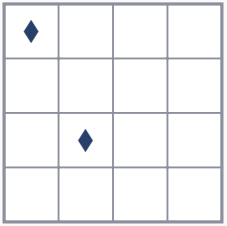
Backtracking on 4-queens



Column 1: place the queen on row 1.

© 2026 Bogomil Stoyanov

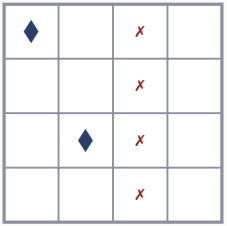
Backtracking on 4-queens



Column 2: rows 1 and 2 attack it, so row 3.

© 2026 Bogomil Stoyanov

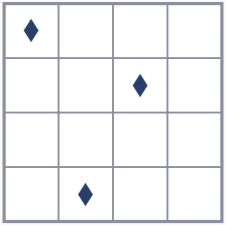
Backtracking on 4-queens



Column 3: **every row is attacked.** Dead end: back up and move the queen in column 2.

© 2026 Bogomil Stoyanov

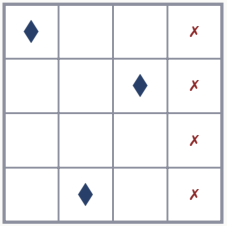
Backtracking on 4-queens



Column 2 to row 4; column 3 can now take row 2.

© 2026 Bogomil Stoyanov

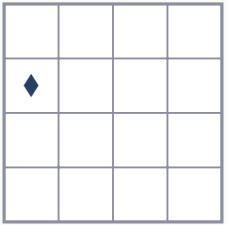
Backtracking on 4-queens



Column 4: **every row attacked again.** Column 3 has no other option either, and neither does column 2, so we back up all the way to column 1.

© 2026 Bogomil Stoyanov

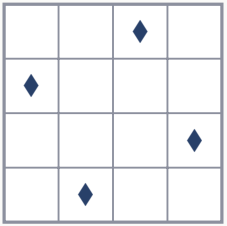
Backtracking on 4-queens



Column 1 to row 2. Start again from there.

© 2026 Bogomil Stoyanov

Backtracking on 4-queens



Solution: rows $2, 4, 1, 3$**.** Two dead ends and one deep backtrack. For $n = 4$ that is cheap; the cost grows very quickly.

© 2026 Bogomil Stoyanov

Ordering: which variable, which value

© 2026 Bogomil Stoyanov

Ordering heuristics

Minimum remaining values (MRV): assign the variable with the *fewest* legal values left.

- “Fail first.” If a variable is going to fail, find out now, at the top of the tree, not after building a subtree under it.

© 2026 Bogomil Stoyanov

Ordering heuristics

Minimum remaining values (MRV): assign the variable with the *fewest* legal values left.

- "Fail first." If a variable is going to fail, find out now, at the top of the tree, not after building a subtree under it.

Degree heuristic: break MRV ties by taking the variable constraining the most unassigned others.

- On our graph that is F, with degree 5.

© 2026 Bogomil Stoyanov

Ordering heuristics

Minimum remaining values (MRV): assign the variable with the *fewest* legal values left.

- "Fail first." If a variable is going to fail, find out now, at the top of the tree, not after building a subtree under it.

Degree heuristic: break MRV ties by taking the variable constraining the most unassigned others.

- On our graph that is *F*, with degree 5.

Least constraining value, for the chosen variable, try the value that rules out the *fewest* options for its neighbours.

- "Fail last": we only need one solution.

© 2026 Bogomil Stoyanov

Ordering heuristics

Minimum remaining values (MRV): assign the variable with the *fewest* legal values left.

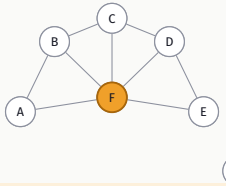
- "Fail first." If a variable is going to fail, find out now, at the top of the tree, not after building a subtree under it.

Degree heuristic: break MRV ties by taking the variable constraining the most unassigned others.

- On our graph that is *F*, with degree 5.

Least constraining value, for the chosen variable, try the value that rules out the *fewest* options for its neighbours.

- "Fail last": we only need one solution.



The first two run one way and the third runs the other: fail fast on variables, fail slow on values. No contradiction: every variable must be tried, but only one value per variable needs to work.

© 2026 Bogomil Stoyanov

Ordering heuristics

Minimum remaining values (MRV): assign the variable with the *fewest* legal values left.

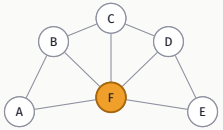
- "Fail first." If a variable is going to fail, find out now, at the top of the tree, not after building a subtree under it.

Degree heuristic: break MRV ties by taking the variable constraining the most unassigned others.

- On our graph that is *F*, with degree 5.

Least constraining value, for the chosen variable, try the value that rules out the *fewest* options for its neighbours.

- "Fail last": we only need one solution.



The first two run one way and the third runs the other: fail fast on variables, fail slow on values. No contradiction: every variable must be tried, but only one value per variable needs to work.

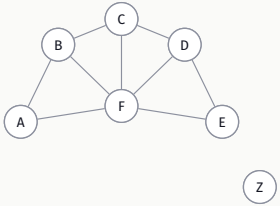
MRV and the degree heuristic together take n-queens from $n \approx 25$ to about $n \approx 1000$.

© 2026 Bogomil Stoyanov

Constraint propagation

© 2026 Bogomil Stoyanov

Forward checking

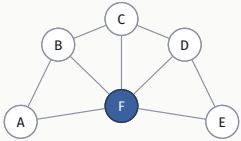


	A	*B*	*C*	*D*	*E*	*F*	*Z*
start							

Every domain starts as {■■■}. Shown here already reduced by *F*'s assignment where relevant.

© 2026 Bogomil Stoyanov

Forward checking

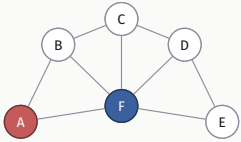




Assign $F=$ ■. Forward checking deletes ■ from all five neighbours.

© 2026 Bogomil Stoyanov

Forward checking

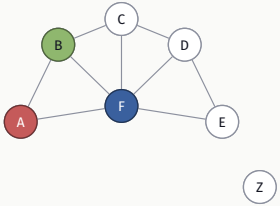




Assign A=■. Delete ■ from B, whose domain is now a **singleton**.

© 2026 Bogomil Stoyanov

Forward checking

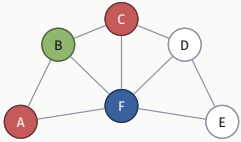


	A	B	C	D	E	F	Z
B=							

So MRV picks B next, and there is nothing to decide: B=.

© 2026 Bogomil Stoyanov

Forward checking

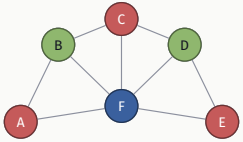


	A	B	C	D	E	F	Z
C=	■	■	■	■	■■	■	■■■

Which forces C=■…

© 2026 Bogomil Stoyanov

Forward checking



z



…and then *D* and *E*. **After two real decisions the rest was propagation, not search.**

© 2026 Bogomil Stoyanov

Arc consistency goes further

AC-3(CSP)

```
1 queue ← all arcs (X_i, X_j)
2 while queue not empty
3       (X_i, X_j) ← POP(queue)
4       if REVISE(X_i, X_j)
5           if D_i is empty then return failure
6           for each X_k neighbour of X_i
7               add (X_k, X_i) to queue          ▷ re-check
```

REVISE deletes every value of D_i that has no consistent partner left in D_j, and reports whether it deleted anything. Cost $O(n^2 d^3)$.

© 2026 Bogomil Stoyanov

Arc consistency goes further

AC-3(CSP)

```
1  queue ← all arcs (X_i, X_j)
2  while queue not empty
3      (X_i, X_j) ← POP(queue)
4      if REVISE(X_i, X_j)
5          if D_i is empty then return failure
6          for each X_k neighbour of X_i
7              add (X_k, X_i) to queue          ▷ re-check
```

REVISE deletes every value of D_i that has no consistent partner left in D_j, and reports whether it deleted anything. Cost $O(n^2 d^3)$.

Forward checking only looks at the neighbours of the variable just assigned. Line 6 is the difference: when a domain shrinks, AC-3 goes back and re-checks everything pointing *at* it.

© 2026 Bogomil Stoyanov

Arc consistency goes further

AC-3(CSP)

```
1  queue ← all arcs (X_i, X_j)
2  while queue not empty
3      (X_i, X_j) ← POP(queue)
4      if REVISE(X_i, X_j)
5          if D_i is empty then return failure
6          for each X_k neighbour of X_i
7              add (X_k, X_i) to queue          ▷ re-check
```

REVISE deletes every value of D_i that has no consistent partner left in D_j, and reports whether it deleted anything. Cost $O(n^2 d^3)$.

Forward checking only looks at the neighbours of the variable just assigned. Line 6 is the difference: when a domain shrinks, AC-3 goes back and re-checks everything pointing *at* it.

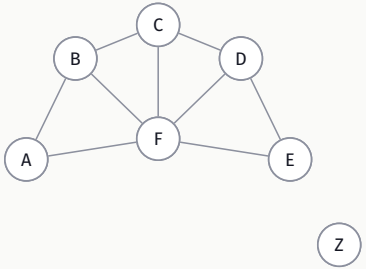
On the previous slide, forward checking made B a singleton and then stopped. Arc consistency would have carried on down the chain immediately (B, then C, then D, then E), **finishing the problem after only two assignments** instead of discovering it one step at a time.

© 2026 Bogomil Stoyanov

Structure

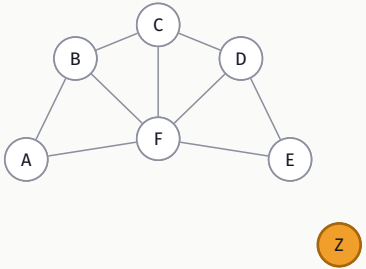
© 2026 Bogomil Stoyanov

Read the graph, not the problem



© 2026 Bogomil Stoyanov

Read the graph, not the problem

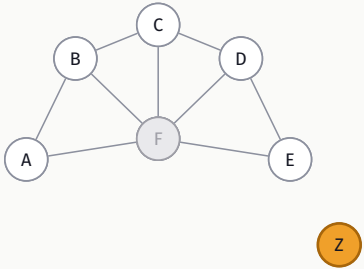


Independent subproblems.
Z touches nothing. Solving it separately turns d^7 into $d^6 + d^1$.

In general $n = 80$, $d = 2$ in components of 20 goes from 4 billion years to **0.4 seconds**.

© 2026 Bogomil Stoyanov

Read the graph, not the problem

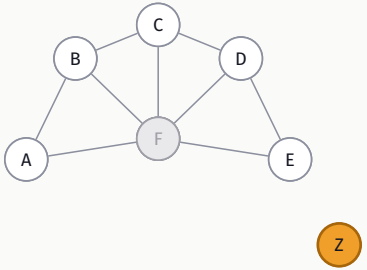


Independent subproblems.
Z touches nothing. Solving it separately turns d^7 into $d^6 + d^1$.
In general $n = 80$, $d = 2$ in components of 20 goes from 4 billion years to **0.4 seconds**.

Trees are easy.
Delete the hub F and what remains is the path $A–B–C–D–E$: a tree.
Any tree-structured CSP is solvable in $O(nd^2)$ with no search at all: root it, make each arc consistent bottom-up, then assign top-down.

© 2026 Bogomil Stoyanov

Read the graph, not the problem



Independent subproblems.
Z touches nothing. Solving it separately turns d^7 into $d^6 + d^1$.

In general $n = 80$, $d = 2$ in components of 20 goes from 4 billion years to **0.4 seconds**.

Trees are easy.
Delete the hub F and what remains is the path A–B–C–D–E: a tree. Any tree-structured CSP is solvable in $O(nd^2)$ with no search at all: root it, make each arc consistent bottom-up, then assign top-down.

Cutset conditioning.
So guess F (d ways), and each time solve a tree. $O(d^c \cdot (n - c)d^2)$ for a cutset of size c.

Here $c = 1$: **three guesses and three tree solves**, and the whole problem is done.

© 2026 Bogomil Stoyanov

Local search: min-conflicts

© 2026 Bogomil Stoyanov

Drop the partial assignments entirely

MIN-CONFLICTS(CSP, MAXSTEPS)

```
1 current ← a complete random assignment
2 for i = 1 to maxSteps
3     if current is a solution then return it
4     X ← a random conflicted variable
5     v ← value of X minimising conflicts
6     set X = v                    ▷ ties broken randomly
7 return failure                   ▷ caller restarts
```

© 2026 Bogomil Stoyanov

Drop the partial assignments entirely

MIN-CONFLICTS(CSP, MAXSTEPS)

```
current ← a complete random assignment
for i = 1 to maxSteps
    if current is a solution then return it
    X ← a random conflicted variable
    v ← value of X minimising conflicts
    set X = v                          ▷ ties broken randomly
return failure                         ▷ caller restarts
```

Backtracking builds a solution one variable at a time and never breaks a constraint. Min-conflicts starts with *everything* assigned and everything broken, then repairs.

For n-queens: one queen per column always, so no queen can ever share a column. Only rows and diagonals can conflict.

© 2026 Bogomil Stoyanov

Drop the partial assignments entirely

MIN-CONFLICTS(CSP, MAXSTEPS)

```
current ← a complete random assignment
for i = 1 to maxSteps
    if current is a solution then return it
    X ← a random conflicted variable
    v ← value of X minimising conflicts
    set X = v                          ▷ ties broken randomly
return failure                         ▷ caller restarts
```

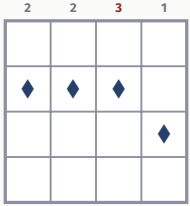
Backtracking builds a solution one variable at a time and never breaks a constraint. Min-conflicts starts with *everything* assigned and everything broken, then repairs.

For n-queens: one queen per column always, so no queen can ever share a column. Only rows and diagonals can conflict.

> Note there is no notion of “backtrack”, and therefore no stack, no partial assignment, and **memory independent of n**.

© 2026 Bogomil Stoyanov

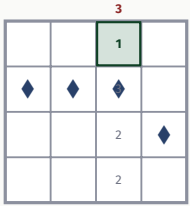
Min-conflicts on 4-queens



A random start, one queen per column. The numbers above each column count the conflicts that queen is in: $2, 2, 3, 1$.

© 2026 Bogomil Stoyanov

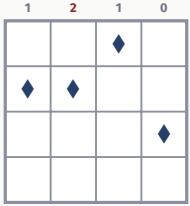
Min-conflicts on 4-queens



Column 3 is the worst. Count the conflicts each row in that column would give: 1, 3, 2, 2, so **row 1 is best**.

© 2026 Bogomil Stoyanov

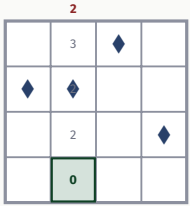
Min-conflicts on 4-queens



Move it. Four conflicting pairs became two. Now column 2 is the worst.

© 2026 Bogomil Stoyanov

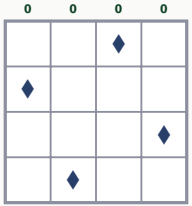
Min-conflicts on 4-queens



Its four options cost 3, 2, 2, 0, so **row 4 is free**.

© 2026 Bogomil Stoyanov

Min-conflicts on 4-queens



Solution: rows 2, 4, 1, 3**: two moves.** The same answer backtracking reached earlier, with two dead ends on the way.

© 2026 Bogomil Stoyanov

Min-conflicts can get stuck

Deterministic tie-breaking loops. If you always take the *first* maximally-conflicted queen and its *first* best row, the search can cycle through the same states forever. It will happily do this for the full `maxSteps`.

Fix: break both ties **randomly**. Those two `random`s in the pseudocode do real work; without them the search can cycle.

© 2026 Bogomil Stoyanov

Min-conflicts can get stuck

Deterministic tie-breaking loops. If you always take the *first* maximally-conflicted queen and its *first* best row, the search can cycle through the same states forever. It will happily do this for the full `maxSteps`.

Fix: break both ties **randomly**. Those two `random`s in the pseudocode do real work; without them the search can cycle.

Local minima. A state can have conflicts while every single move makes things worse or equal. 4-queens has only 2 solutions in 256 states, so this is common at small n.

Fix: **random restart** after `maxSteps`. That is the outer loop the pseudocode's last line hands back to.

© 2026 Bogomil Stoyanov

Min-conflicts can get stuck

Deterministic tie-breaking loops. If you always take the *first* maximally-conflicted queen and its *first* best row, the search can cycle through the same states forever. It will happily do this for the full `maxSteps`.

Fix: break both ties **randomly**. Those two `random`s in the pseudocode do real work; without them the search can cycle.

Local minima. A state can have conflicts while every single move makes things worse or equal. 4-queens has only 2 solutions in 256 states, so this is common at small n.

Fix: **random restart** after `maxSteps`. That is the outer loop the pseudocode's last line hands back to.

Both fixes are randomness. A completely deterministic local search has no way out of a state whose neighbours are all worse.

© 2026 Bogomil Stoyanov

Scaling to $n = 10\,000$

Min-conflicts solves n-queens in a number of steps that is *almost independent of n*: around 50 steps whether n is 100 or 10^6.

But a naive step is $O(n)$: recounting every queen's conflicts from scratch. At $n = 10\,000$ that is the whole cost.

© 2026 Bogomil Stoyanov

Scaling to $n = 10\,000$

Min-conflicts solves n-queens in a number of steps that is *almost independent of* n: around 50 steps whether n is 100 or 10^6.

But a naive step is $O(n)$: recounting every queen's conflicts from scratch. At $n = 10\,000$ that is the whole cost.

So keep counts incrementally. One queen per column means a queen is identified by its column, and a conflict can only be a shared row or a shared diagonal:

© 2026 Bogomil Stoyanov

Scaling to $n = 10\,000$

Min-conflicts solves n-queens in a number of steps that is *almost independent of* n: around 50 steps whether n is 100 or 10^6.

But a naive step is $O(n)$: recounting every queen's conflicts from scratch. At $n = 10\,000$ that is the whole cost.

So keep counts incrementally. One queen per column means a queen is identified by its column, and a conflict can only be a shared row or a shared diagonal:

STATE

```
1 row[c]      ▷ the queen in column c
2 perRow[r]   ▷ queens on row r
3 perD1[c-r]  ▷ queens on each ↘
4 perD2[c+r]  ▷ queens on each ↗
```

© 2026 Bogomil Stoyanov

Scaling to $n = 10\,000$

Min-conflicts solves n-queens in a number of steps that is *almost independent of* n: around 50 steps whether n is 100 or 10^6.

But a naive step is $O(n)$: recounting every queen's conflicts from scratch. At $n = 10\,000$ that is the whole cost.

So keep counts incrementally. One queen per column means a queen is identified by its column, and a conflict can only be a shared row or a shared diagonal:

STATE

```
1 row[c]      ▷ the queen in column c
2 perRow[r]   ▷ queens on row r
3 perD1[c-r]  ▷ queens on each ↘
4 perD2[c+r]  ▷ queens on each ↗
```

A move updates six counters, so finding conflicts becomes $O(1)$ instead of $O(n)$. The three tally arrays are the entire trick.

© 2026 Bogomil Stoyanov

Summary

© 2026 Bogomil Stoyanov

Summary

Technique	What it does	What it buys
Backtracking	one variable per level	$n!d^n \to d^n$
MRV + degree	fail first	$n{\approx}25 \to n{\approx}1000$
Least constraining value	fail last	fewer retries
Forward checking	prune neighbours' domains	detects dead ends early
Arc consistency (AC-3)	propagate until stable	often solves outright
Independent components	split the graph	$d^n \to \sum d^{n_i}$
Tree structure	no search needed	$O(nd^2)$
Cutset conditioning	guess a cutset, solve trees	$O(d^c(n-c)d^2)$
Min-conflicts	repair a complete assignment	$n = 10\,000$

© 2026 Bogomil Stoyanov

Summary

Technique	What it does	What it buys
Backtracking	one variable per level	$n!d^n \to d^n$
MRV + degree	fail first	$n{\approx}25 \to n{\approx}1000$
Least constraining value	fail last	fewer retries
Forward checking	prune neighbours' domains	detects dead ends early
Arc consistency (AC-3)	propagate until stable	often solves outright
Independent components	split the graph	$d^n \to \sum d^{n_i}$
Tree structure	no search needed	$O(nd^2)$
Cutset conditioning	guess a cutset, solve trees	$O(d^c(n-c)d^2)$
Min-conflicts	repair a complete assignment	$n = 10\,000$

Every row follows from one thing: being able to *see inside the state*. Last week's algorithms could do none of it.

© 2026 Bogomil Stoyanov